National Taiwan University
Special Project for Bachelor Degrees Student

Precision Planning of a Robotic Arm Using Computer Vision

Lifting a Cup of Coffee Using SCARA Controlled by OpenCV

Author: Tsung-Wun Wang (Student ID: B05502097)
Professor: Han-Pang Huang
Senior: Yao Wu

Index Terms—computer vision, robotic arm, robot programming.
Report Due Date: 2nd, January, 2019

Contents

Contents · p. 02

I. INTRODUCTION · p. 04

II. MATERIALS AND METHODS · · · · · · · · · · · · · · · · · · p. 04

1. Purpose · p. 04
2. Equipment · p. 04
2.1 SCARA - Robot Arm's Movement - teaching mode · · · · · · · · · · p. 05
2.2 Gripper · p. 06
2.2.1 Gripper - Introduction
2.2.2 Gripper – CAD Drawing
2.2.3 Gripper – Programming
2.3 OpenCV · p. 15
2.3.1 Simple Image Analysis
2.3.2 Real-time Image Processing
2.4 SCARA – Program to integrate Real-time Image Processing · · · · · · p. 17
2.4.1 SCARA – Trajectory planning

III. EXPERIMENTAL SETUP AND RESULTS · · · · · · · · · · · · p. 19

1. Operating Procedures · p. 19
1.1 maker_func_ins_macro
2. Network Connection · p. 20
3. Initializing Parameters-Class Constructor in C++ · · · · · · · · · · p. 21
4. Demo · p. 21
5. Problems · p. 21

IV. CONCLUSIONS · p. 22

V. DISCUSSION · p. 23

1. Six-Axis articulated robots · p. 23
2. Camera Calibration and Distortion Correction · · · · · · · · · · · p. 24
3. Find a better method of setting environmental parameters · · · · · · · p. 24

REFERENCES . . . . . p. 24

APPENDICES . . . . . p. 25

Appendix A: Testing The Environment . . . . . p. 25

Appendix B: OpenCV + SCARA + Trajectory Planning . . . . . p. 27

Appendix B-S0: main.cpp . . . . . p. 27

Appendix B-H1: connection.h . . . . . p. 29

Appendix B-S1: connection.cpp . . . . . p. 30

Appendix B-H2: MotionPlanning.h . . . . . p. 33

Appendix B-S2: MotionPlanning.cpp . . . . . p. 34

Appendix B-H3: ReadCoordinate.h . . . . . p. 38

Appendix B-S3: ReadCoordinate.cpp . . . . . p. 38

Appendix B-H4: WebCam.h . . . . . p. 39

Appendix B-S4: WebCam.cpp . . . . . p. 41

I. INTRODUCTION

Robotic Automation has been gradually changing the world. Because robots are efficient, intelligent and precise, they become popular among people, especial for those who specialize in industrial automation. With robots, plenty of tasks and applications can be performed and completed faster, and the production and profit margin will increase

Today, most convenience stores provide coffee service. However, those who make coffee are replaced only by few robotic arms in convenience stores even in this modern world. Although robots have flexibility with being capable of performing a variety of tasks and applications, some problems may arise if precision level does not meet expectations. For example, if robotic arms have no ability to keep the angle of inclination constant, ideally zero in most case, the coffee in the cup will run down the side. To solve this problem, we need to add a new subsystem to detect whether the coffee cup is tilted, such as sensors, cameras, or any other device. In this article, I will try and design a computer vision system, which is then carried out and calibrate the motion of the robotic arm, in order to make all coffee cups moved by the robotic arm stable.

II. MATERIALS AND METHODS

1. Purpose

My task is to investigate the possibility to integrate the computer vision into a robotic arm that are capable of moving a cup of coffee, from a pick up location to a landing.

2. Equipment

- SCARA
- WebCam
- Gripper
- PC + charging cable + Ethernet Cable
- Cellphone
- Coffee cup
- User Manual
- Pencil and Pen

2.1 SCARA - Robot Arm’s Movement - teaching mode

In my first step of my project, I learn the method of controlling this robotic arm. There are two modes of movement: teaching mode and auto mode. The movement in teaching mode can be controlled by a teach box, and we have different types of coordinates to choose from, such as world coordinates and joint coordinates.

Note that I use the movement in auto mode in my final demo.

Figure 1: SCARA Controller

Figure 2: A teach box is a control box and is used to control the robot step by step

2.2 Gripper

2.2.1 Gripper - Introduction

To let the gripper grabs the cup successfully, we have to make the motors run without any accidents. Using motion controllers is a good choice because they are the brains of any motion control system. In feedback-based systems, they take an input command from the user, compare it with a feedback signal from the motor, and take corrective action to bring the output (or actual position) and input (or desired position) in line with one another, ideally with little or no error.

A motion controller also creates the trajectories that the motors follow in order to meet the desired commands. Also called motion profiles, a profile is a sequence of position commands versus time. This tells the motor where to position the load and how fast it must do so. The motion controller uses the trajectories it creates to generate the proper torque commands. These torque commands are then sent to the drive which powers the motor.

Here is our motion controller. Its model number is **MCDC 3006 S**.

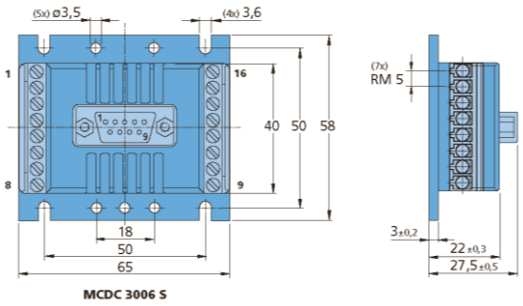
	
Product: Motion Controllers Number: Series MCDC 3006 S	Scale reduced Drawing

Table 3: Product Appearance [2]

Faulhaber: Motion Controller V2.5 (MCDC 3006 Series)

V2.5, 4-Quadrant PWM, with RS232 or CAN interface

FAULHABER Motion Controllers are easy to use single axis positioning controllers tailor made for the control of FAULHABER DC, Brushless, and linear motors.

For high resolution dynamic positioning applications, or high precision speed control applications they are available with serial or CAN interfaces and can be configured the free and easy to use Motion Manager software.

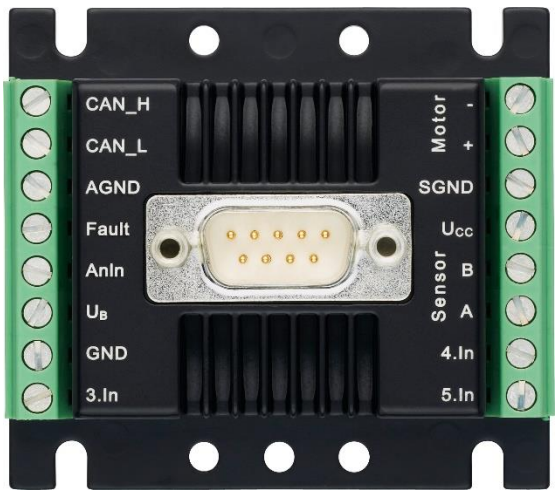
Features and Benefits:

- Compact design
- Precise position and speed control
- Serial or CAN interfaces
- Many standard operating modes
- Integrated Motor Protection functions
- Simple USB configuration
- Easy to use software

Specifications:

- Control Type:1,2,3
- Functionality: 4 -Q/PWM
- Interface: RS232/CAN
- Type of motors: DC
- UB: 12 ... 30
- Imax:10

Table 4: Description of the Motion Controller V2.5 (MCDC 3006 Series) [3]



Connection

No.	Function	No.	Function
1	TxD / CAN_H	9	5. In
2	RxD / CAN_L	10	4. In
3	AGND	11	Ch A
4	Fault	12	Ch B
5	AnIn	13	U_{CC}
6	U_B	14	SGND
7	GND	15	Mot +
8	3. In	16	Mot -

D-SUB connector

RS-232		CAN	
No.	Function	No.	Function
2	RxD	2	CAN_L
3	TxD	3	GND
5	GND	5	-
7	-	7	CAN_H

List of abbreviations

Abbreviation	Meaning
AnIn	Analogue Input
AGND	Analogue Ground
CAN	Controller Area Network
CAN_L	CAN-Low
CAN_H	CAN-High
CF	Controller with CANopen interface (Faulhaber channel)
CLK	Clock
CO	Controller with CANopen interface acc. to CiA 402
CS	Chip Select Cable
Data	Data cable
DigIn	Digital input
DigOut	Digital output
EMC	Electromagnetic compatibility
ESD	Electrostatic discharge
FAULT	Fault output
GND	Ground
PLC	Programmable Logic Controller
PWM	Pulse Width Modulation
RS	Controller with serial RS232 interface
RxD	Receive data
TTL	Transistor Transistor Logic
TxD	Transmit data

Figure 5: Connection Information [2]

2.2.2 Gripper – CAD Drawing

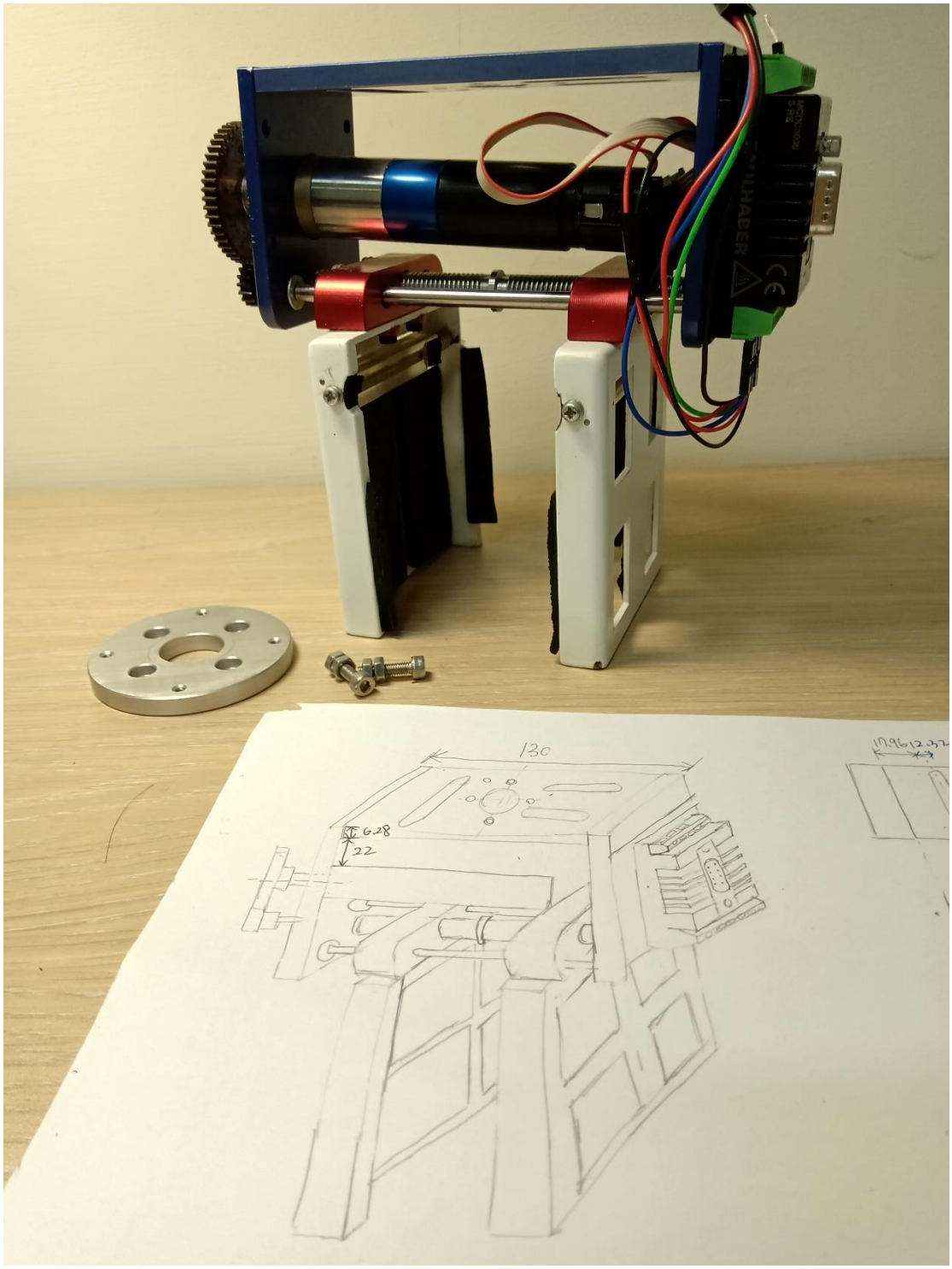


Figure 6: The original gripper before processed, and my 3D sketch.

Because the gripper should be installed on SCARA on my final demo, there are three components that need to be replaced. First I design and make parts in Solidworks.

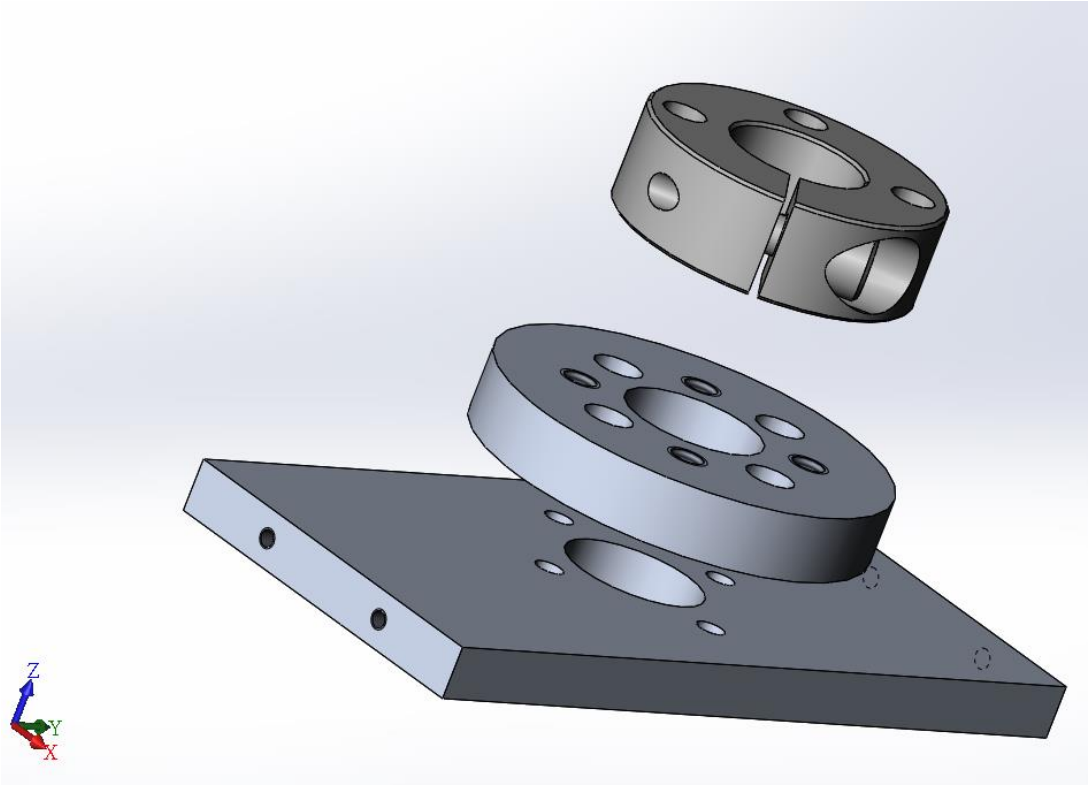


Figure 7: The three components from top to bottom are: set collar, circular plate, rectangular plate. This figure is drawn in Solidworks.

From Figure 7, we see three components from top to bottom: set collar, circular plate, rectangular plate. We replace top blue part in Figure 6 with a rectangular plate, the bottom one in Figure 7. A set collar is the product we buy from MISUMI, a manufacturing company, because it is a standard part. There are four tapped holes and four straight holes on the circular plate. The tapped holes are used to connect the set collar, and the straight holes are used to connect the rectangular plate. You can find more details in the following figures.

固定環 —開縫型—

—3孔型/3攻牙(粗螺紋)型—

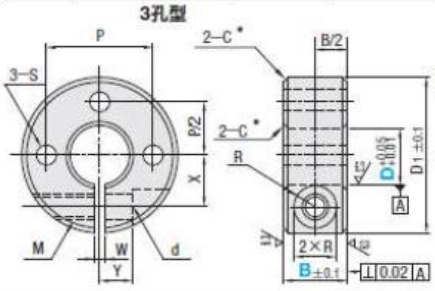
CAD資料夾名稱：03_Set_Colla

■3孔型
■3攻牙(粗螺紋)型

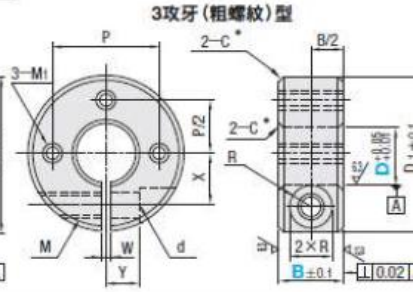
Type 3孔	Type 3攻牙	M材質	S表面處理	A附屬品
SCST	SCSTR	相當於S45C	染黑處理	內六角螺栓 1pcs
PSCST	PSCSTR		鍍無電解鎳	M相當於SCM435
SSCST	SSCSTR	相當於SUS304	—	M相當於SUS304

*D6 ~D25 : C0.5
D30~D50 : C1

3孔型



3攻牙(粗螺紋)型



型式 Type	D	B 選擇	D1	M (粗螺紋)	d	R	W	X	Y	P	S	3孔型 NTS基準單價 SCST	PSCST	SSCST	3攻牙型 M1 (粗螺紋)	NTS基準單價 SCSTR	PSCSTR	SSCS
(3孔型) SCST PSCST SSCST (3攻牙型) SCSTR PSCSTR SSCSTR	6	*8	20	M3	3.4	3	1	6	4	12	3.5						—	
		*10		M4	4.5	3.75				13			—		M3			
	8	*8	25	M3	3.4	3		8		14								
		*10		M4	4.5	3.75				18	4.5				M4			
		15	30	M6	6.6	6		9	5				—	—	—	—	—	—
	10	*8		M3	3.4	3				20	5.5				M5			—
		*10		M4	4.5	3.75												
		12		M5	5.5	4.5												
		15	35	M6	6.6	6			6	24	6.6			—	—	—	—	—
	12	8	30	M3	3.4	3		10										
		*10		M4	4.5	3.75			5	22	5.5				M5			
		*12		M5	5.5	4.5												
		15	35	M6	6.6	6		11	6	24	6.6				—	—	—	—
	13	*10	30	M4	4.5	3.75		10	5	22	4.5				M4			
		12	34	M5	5.5	4.5	1.5	11		23	5.5		—	—	—	—	—	—
		15	35	M6	6.6	6			6	24								
	15	*10	34	M4	4.5	3.75		11.5		23	4.5				M4			
		*12	35	M5	5.5	4.5		12		26	5.5				M5			
		*15	40	M6	6.6	6		13	8	28	6.6				M6			
	16	10	35	M5	5.5	4.5		12	6	24	4.5				—	—	—	—
		*12								26	5.5				M5			
		*15	40	M6	6.6	6		13	8	28	6.6				M6			
	20	*10		M5	5.5	4.5		14	7		4.5				M4			
		*12								30	5.5				M5			
		*15	45	M6	6.6	6		15	8	34	6.6				M6			
	25	*12	45	M5	5.5	4.5		17	7	36	5.5				M5			
		*15	50					18	10	38								

・3攻牙型僅限*符號尺寸

Figure 8: The set collar from the MISUMI catalog. [4]

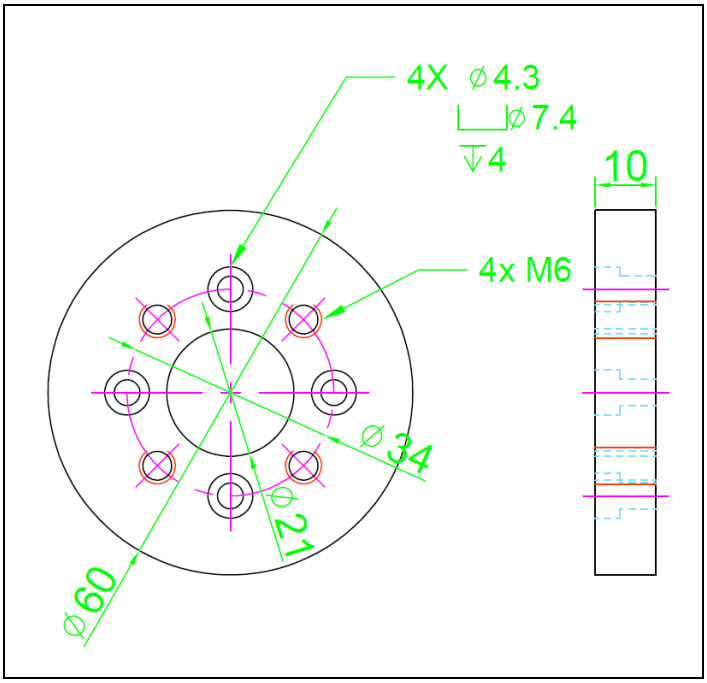


Figure 9: Engineering drawing of the circular plate.

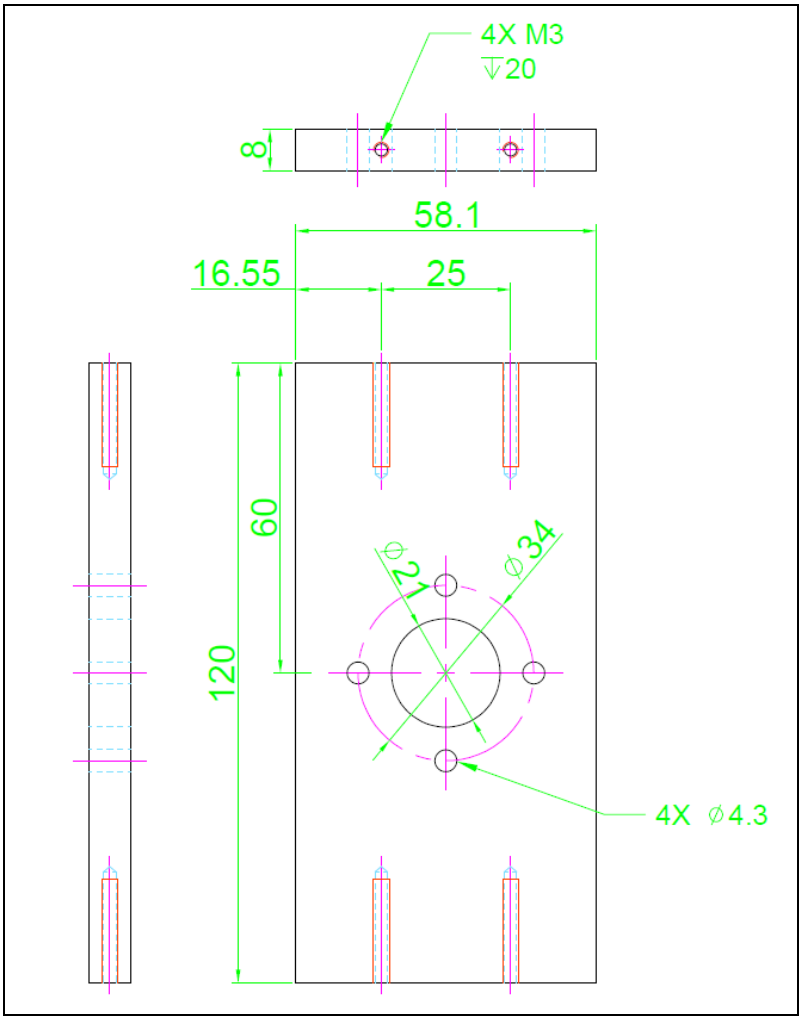


Figure 10: Engineering drawing of the rectangular plate.

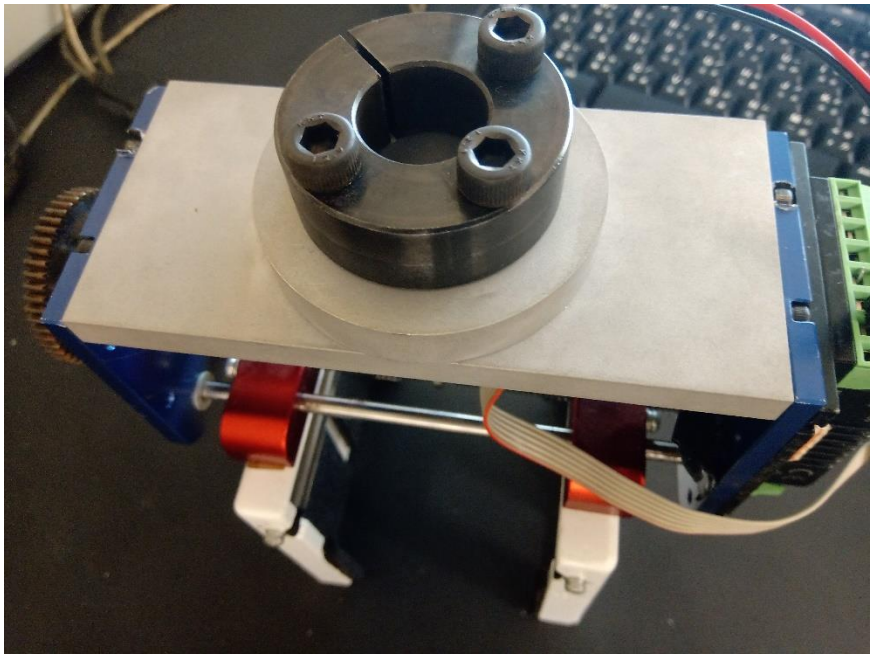


Figure 11: After design and being processed, all parts are assembled together.

2.2.3 Gripper – Programming

We need to program the motion controller first. The operation interface in shown in Figure 12. Then we install electrical wiring using lines and cables, and distribute electric power with power supply.

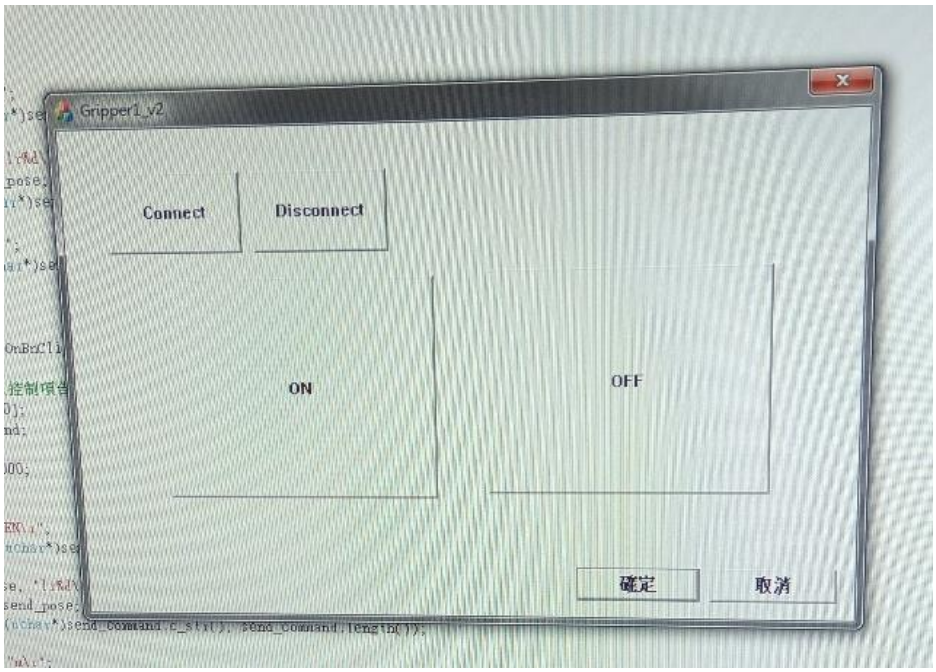


Figure 12: The operation interface of the gripper.

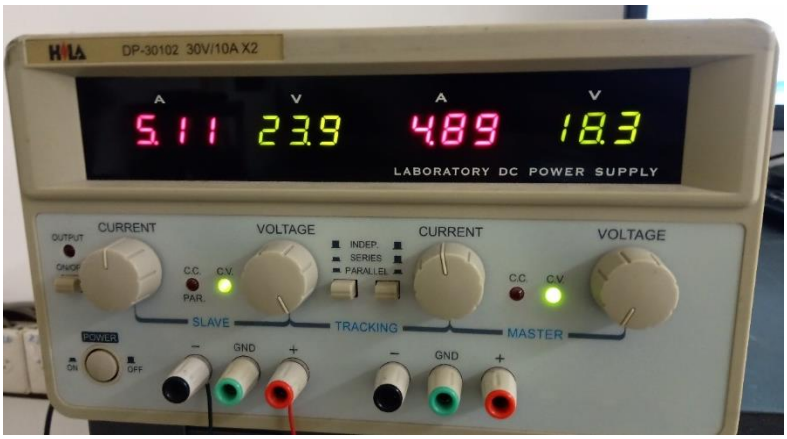


Figure 13: Power Supply

Figure 14: The gripper grab a cup after programed and installed completely.

2.3 OpenCV

2.3.1 Simple Image Analysis

First of all, we observe that the shape of a cup in the top view is circle. So I write the program using C++ and OpenCV to detect any circle in some pictures.

2.3.2 Real-time Image Processing

After detect some simple circles successfully, I connect my PC to my WebCam and rewrite my program in order to detect circles using real-time image processing.

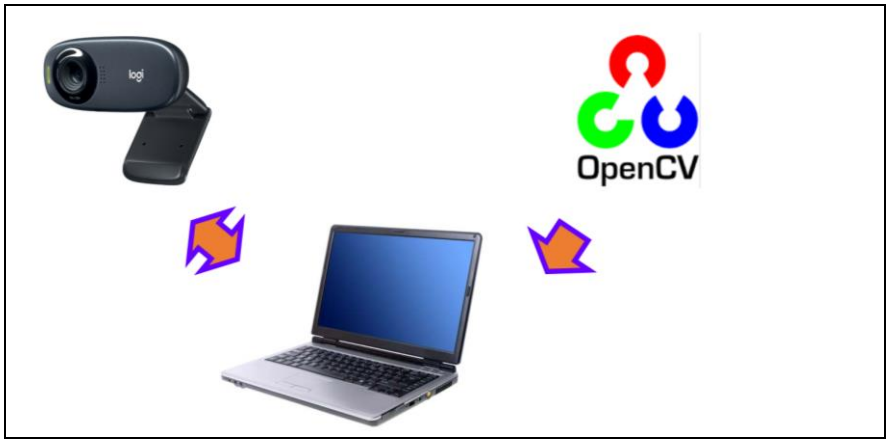


Figure 15: Integrating WebCam into my image-processing project. [5] [6] [7]

Here is the method how Webcam detects circles using C++.

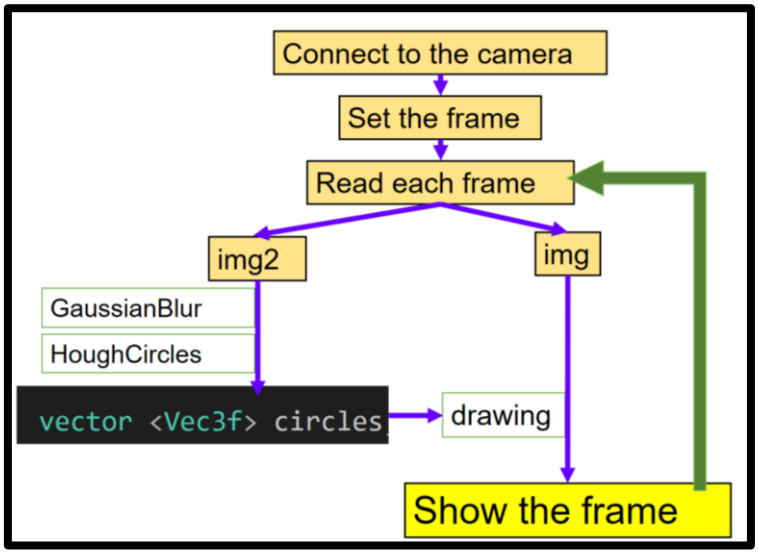


Figure 16: Flow chart of my programming.

The key points in this image-processing project is Gaussian blur and Hough circle transform.

In image processing, a Gaussian blur (also known as Gaussian smoothing) is the result of blurring an image by a **Gaussian function**. It is a widely used effect in graphics software, typically to reduce image **noise** and reduce detail. [8]

A Hough circle transform is an image transform that allows for **circular** objects to be extracted from an image, even if the circle is incomplete. The transform is also selective for circles, and will generally **ignore elongated ellipses**. The transform effectively searches for objects with a high degree of radial symmetry, with each degree of symmetry receiving one "vote" in the search space. By searching a 3D Hough search space, the transform can measure the **centroid** and **radius** of each circular object in an image. [9]

The below YouTube video links shows how the detection works in my project.

https://youtu.be/boGDktq2RrQ

However, some problems appear when the parameters are wrong.

Figure 17: The background color is close to the left circle and the detection does not work.

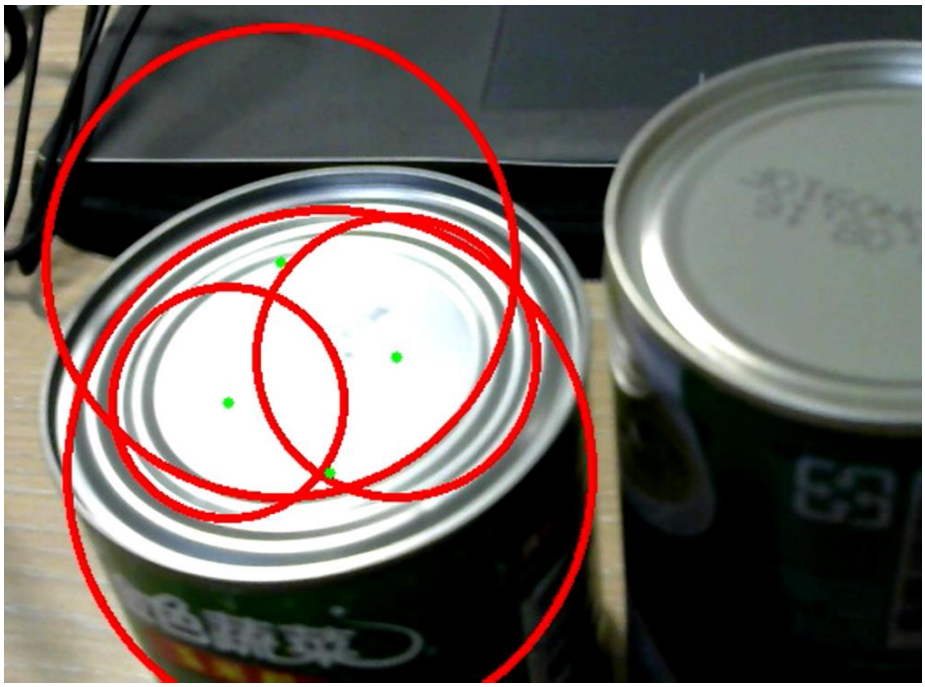


Figure 18: The inclined camera makes the detection out of order.

2.4 SCARA – Program to integrate Real-time Image Processing

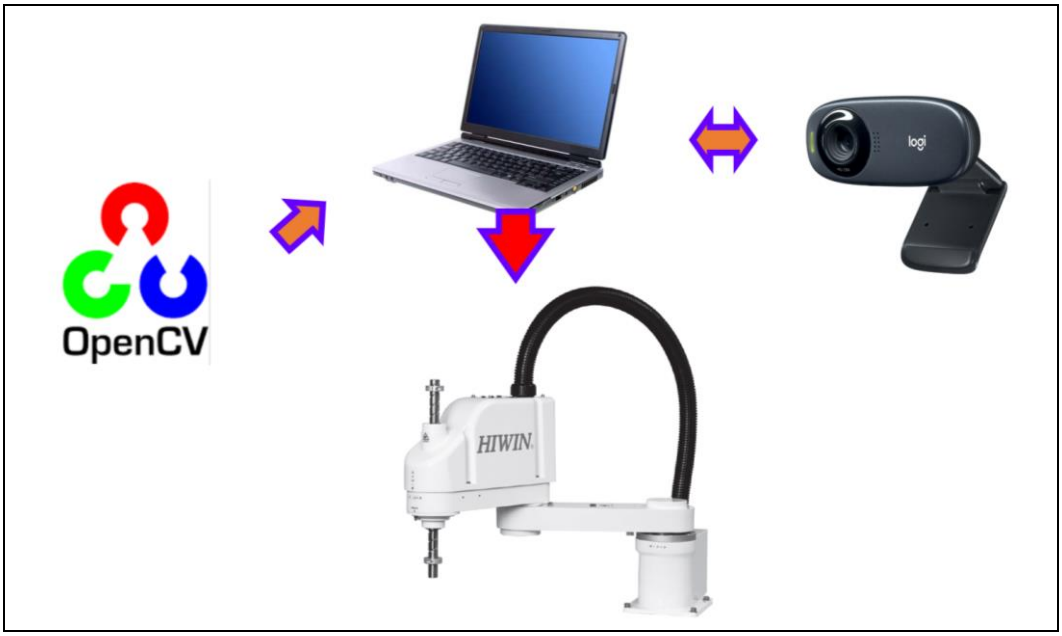


Figure 19: Integration of SCARA and image processing [5] [6] [7] [11]

2.4.1 SCARA – Trajectory planning

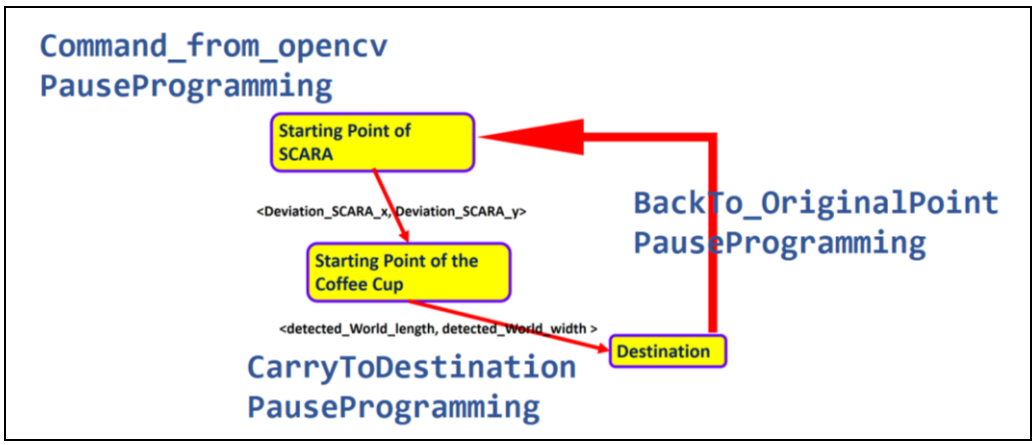


Figure 20: Layout

First the robotic arm initializes at a starting point, and the WebCam will detect where the cup of coffee is and where the cup need to be grabbed to, which is the destination. After finding the two locations, the robotic arm moves onto the cup. After the gripper lifts the cup, the robotic arm will move to the destination and land the cup. Finally, it will return to its starting point, and the WebCam will detect the next cup to grab.

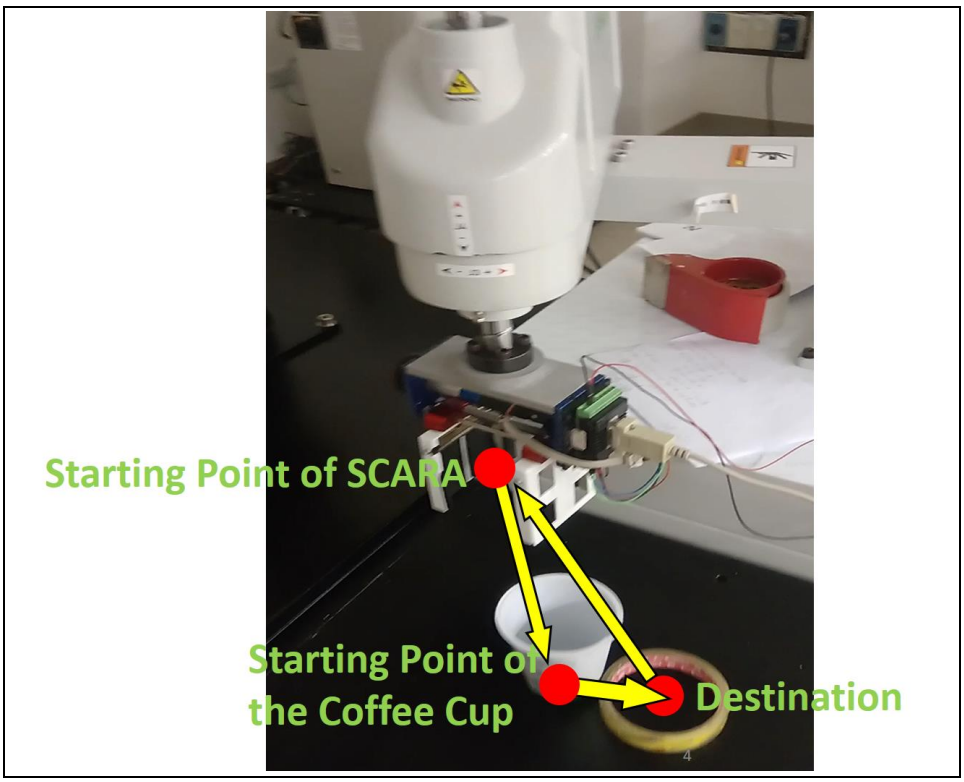


Figure 21: Trajectory planning of the robotic arm

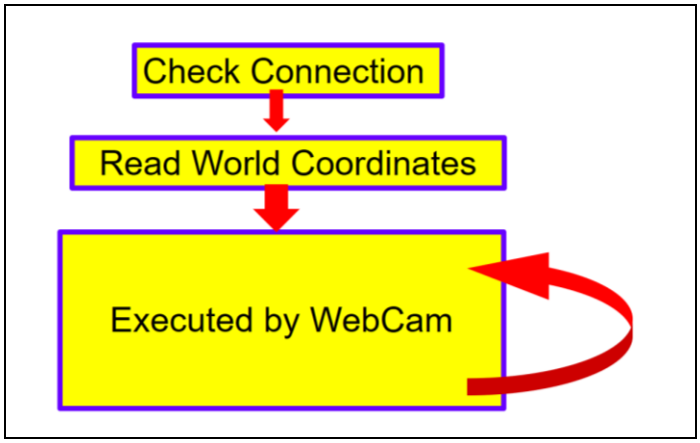


Figure 22: Flow Chart of my final project

The integration is programmed in my final demo, and Figure 22 shows how the system runs. First I will check the connection between my computer and the SCARA controller. Then I read the initial world coordinates before I turn on my WebCam. The SCARA will have the motion, like in Figure 21, again and again. Here is the video without the gripper:

YouTube https://youtu.be/tktUs6oJRnI

III. EXPERIMENTAL SETUP AND RESULTS

1. Operating Procedures

1.1 maker_func_ins_macro

maker_func_ins_macro777.txt - 記事本

檔案(F) 編輯(E) 格式(O) 檢視(V) 說明(H)

```
#1=(R_REG(8503));
#2=(R_REG(8504));
#3=(R_REG(8505));

G01 L1 X#1 Y#2 Z#3 F20000
G04 P300
W_MLC_O(5,1)
PROG_END
```

Figure 23: This file ‘maker_func_ins_macro’ need to be uploaded to the SCARA controller

2. Network Connection

Figure 24: Before check the connection in compiling, I reset the IP address.

3. Initializing Parameters-Class Constructor in C++

Some parameters need to be initialized in my constructor in **WebCam** class:

- Starting point in the form of world coordinates
- Coordinate Transformations
  - **<Deviation_SCARA_x, Deviation_SCARA_y>**
  - **<detected_World_length, detected_World_width >**
- Using 2 bottle caps to set the coordinates in pixels.
- Radius of the coffee cup.
- The parameters in image analysis.

4. Demo

Here is my final project:

Final Project *Click to watch the video!*
YouTube https://youtu.be/avokhlcFqH8

5. Problems

Some problems still occurred. The first problem is that the robotic arm sometimes has no movements along z axis when I give it the trajectory. It seems that the **connection** is weak, so the command will not be received. The second one is that the accuracy is not high enough, and SCARA even breaks the cup! It is a bad news that the **open-loop** control will fail in my project, and the **small open and close stroke** of the gripper also makes the operation fail more frequently. Here is the video how the system fails in some cases:

YouTube https://youtu.be/RBqPE7KWJVU

I find another discussion talking about **Radial Distortion** in image processing, and below is the part of the article.

Radial Distortion

Real cameras use curved lenses to form an image, and light rays often bend a little too much or too little at the edges of these lenses. This creates an effect that distorts the edges of images, so that lines or objects appear more or less curved than they actually are. This is called **radial distortion**, which is the most common type of distortion.

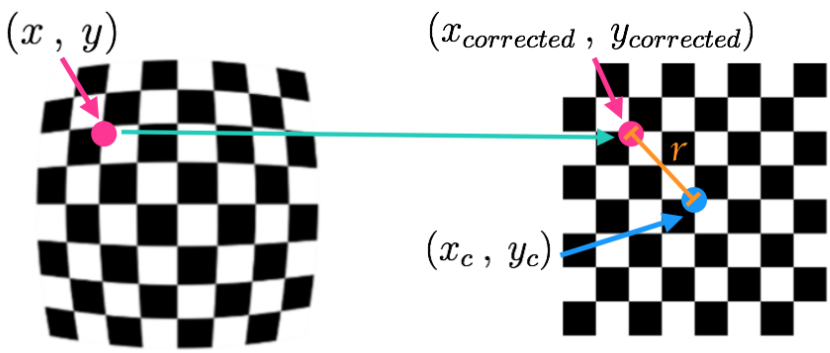


Figure 25: Radial distortion correction technique [10]

IV. CONCLUSIONS

The robotic arm is capable of moving a cup of coffee, from a pick up location to a landing, and this movement can be combined with the computer vision.

V. DISCUSSION

Here I list a few recommendations for future research:

1. Six-Axis articulated robots

Figure 26: HIWIN six-axis articulated robots [11]

The way how SCARA pick up the cup seems a little strange. Because SCARA is a **four-axis** articulated robots, it can only pick up the cup along z axis. It is necessary to implement the project with a six-axis articulated robot to have a better performance.

Figure 27: People grab the coffee in this way. I hope the gripper installed on the robotic arm is also capable of grabbing the coffee from the side.

2. Camera Calibration and Distortion Correction

To deal with the low-accuracy problem, closed-loop control appears to be necessary in this project. With this method, the WebCam should be always turned on until I end the operation. The system should also be implemented with PID control or lead-lag compensator to have a better performance.

Besides, I should learn more skills in image processing, in order to cope with most of the problems, such as the radial distortion mentioned above.

3. Find a better method of setting environmental parameters

In this project, I use two circular object to set the coordinates in pixels. I initialize the parameters in my constructor with two vectors mentioned in Figure 20. However, setting these is an extremely time-consuming process, and most of experimental errors occurs because of the way I find the parameters. So, maybe I need to reconsider an alternative way to increase the accuracy and make the process more efficient.

REFERENCES

[1] A. D. Kulkarni, Computer vision and fuzzy-neural systems. Prentice Hall PTR, 2001, ch. 2 and ch. 6. - http://www.eejournal.ktu.lt/index.php/elt/article/viewFile/13749/7276

[2] Motion Controllers - Series MCDC 3006 S - https://www.faulhaber.com/en/products/series/mcdc-3006-s/

[3] FAULHABER: MOTION CONTROLLER V2.5 (MCDC 3006 SERIES) https://www.e-motionsupply.com/product_p/mcdc-3006-s.htm

[4] 固定環 - 開縫 - 3 孔／3 攻牙 - 型 https://tw.misumi-ec.com/vona2/detail/110300019270/

[5] Picture of my Webcam – https://assets.logitech.com/assets/65486/c310-hd-webcam-refresh.png

[6] picture of a laptop - https://cdn.britannica.com/77/170477-004-B774BDDF.jpg

[7] OpenCV - https://opencv.org/

[8] Gaussian blur - https://en.wikipedia.org/wiki/Gaussian_blur

[9] Hough Circle Transform - https://imagej.net/Hough_Circle_Transform

[10] Advanced Lane Finding https://medium.com/@sujaybabruwad/advanced-lane-finding-project-6476732dcf18

[11] Picture of HIWIN - https://www.hiwin.com/images/rs-605-large.jpg

[12] Picture of grabbing a coffee - https://image.shutterstock.com/image-photo/woman-grab-cup-hot-coffee-260nw-1093868411.jpg

APPENDICES

Appendix A: Testing The Environment

```
1.  #include <iostream>
2.  #include<opencv2/opencv.hpp>
3.  using namespace std;
4.  using namespace cv;
5.
6.  int main()
7.  {
8.      int camera_number = 1;
9.      VideoCapture cap(camera_number + CAP_DSHOW);
10.
11.     if (!cap.isOpened())
12.     {
13.         cout << "Change the camera port number! ";
14.         return -1;
15.     }
16.     namedWindow("main", CV_WINDOW_AUTOSIZE);
17.     Mat img;
18.     Mat img2;
19.     int c;
20.     float radius;
21.     int frame_width = cap.get(CV_CAP_PROP_FRAME_WIDTH);
22.     int frame_height = cap.get(CV_CAP_PROP_FRAME_HEIGHT);
23.     while (true)
24.     {
25.         cap >> img;
26.         imshow("main", img);
27.
28.         cvtColor(img, img2, CV_BGR2GRAY);
29.         GaussianBlur(img2, img2, Size(9, 9), 2, 2);
30.
31.         vector <Vec3f> circles;
32.         HoughCircles(img2, circles, CV_HOUGH_GRADIENT, 1, img2.rows / 8,
    200, 100, 0, 0);
33.
34.         for (size_t i = 0; i < circles.size(); i++)
```

```
35.         {
36.             Point center(cvRound(circles[i][0]),
cvRound(circles[i][1]));
37.             radius = cvRound(circles[i][2]);
38.             cout <<"center = "<<center <<";radius = "<< radius << endl;
39.             circle(img, center, 3, Scalar(0, 255, 0), -1, 8, 0);
40.             circle(img, center, radius, Scalar(0, 0, 255), 3, 8, 0);
41.         }
42.         imshow("main", img);
43.         char c = (char)waitKey(1);
44.         if (c == 27)
45.             break;
46.     }
47.     cap.release();
48.     destroyAllWindows();
49.     return 0;
50. }
```

Appendix B: OpenCV + SCARA + Trajectory Planning

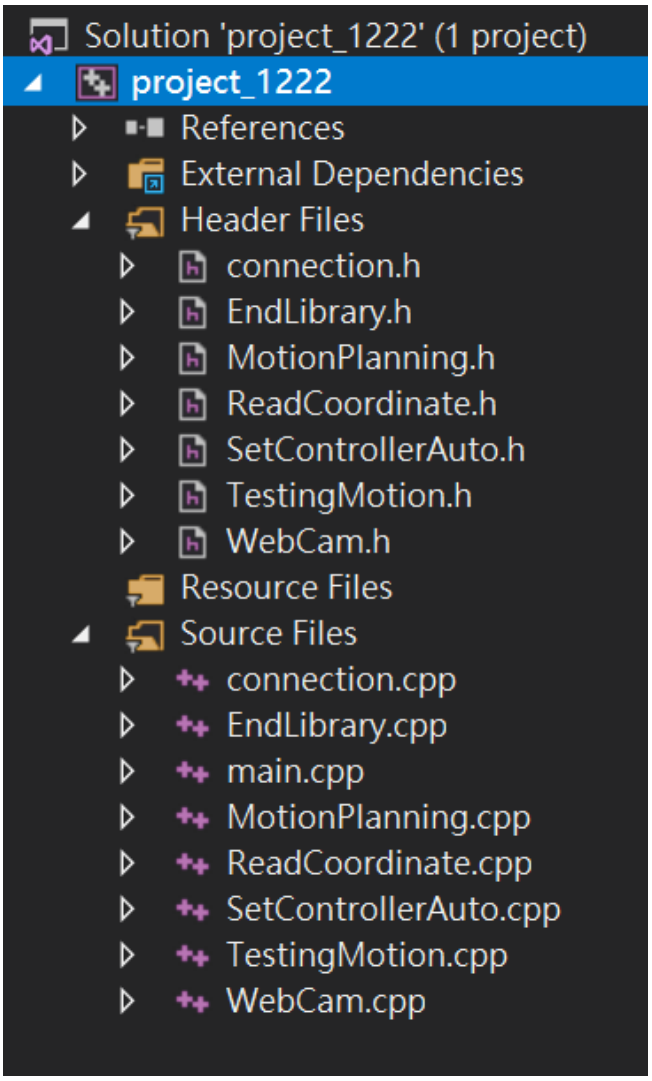


Figure 28: Solution Explorer

Appendix B-S0: main.cpp

```
#define __VC
#define _CRT_SECURE_NO_WARNINGS

#include <iostream>
#include<opencv2/opencv.hpp>
using namespace std;
//using namespace cv;

#include <tchar.h>
#include <stdio.h>
```

```
11. #include <stdlib.h>
12. #include <iostream>
13. #include <cstdlib>
14. #include <Windows.h>
15. #include <fstream>
16. #include <math.h>
17. #include <scif.h>
18.
19.
20. #include "connection.h"
21. #include "SetControllerAuto.h"
22. #include "ReadCoordinate.h"
23. #include "EndLibrary.h"
24. #include "WebCam.h"
25.
26. #include "TestingMotion.h"
27. #include "MotionPlanning.h"
28.
29.
30. int main()
31. {
32.
33.     connection BeConnected;
34.     BeConnected.execution();
35.     Sleep(2000);
36.
37.     //SetControllerAuto C_auto; //Transform - controller mode -  auto
    mode
38.     //C_auto.execution();
39.     //system("Pause");
40.
41.     MotionPlanning m;
42.     m.Check_OriginalPoint();
43.
44.     ReadCoordinate world_coordinate; //Transform  - Read the coordinate
    value
45.     cout << "\n" << world_coordinate.World_X << "\n" <<
    world_coordinate.World_C << "\n" << endl;
```

```
46.     WebCam MyCamera(world_coordinate.World_X, world_coordinate.World_Y,
world_coordinate.World_Z, world_coordinate.World_C);
47.     MyCamera.execution();
48.
49.     //EndLibrary end;
50.     return 0;
51. }
```

Appendix B-H1: connection.h

```
1.  #pragma once
2.  #define __VC
3.  #include "scif.h"
4.  #include "scif_define.h"
5.  #include<iostream>
6.  #include<string.h>
7.
8.  class connection
9.  {
10. public:
11.     DLL_USE_SETTING DllSetting;
12.     const char* IP_Address;
13.     int Maker_ID;
14.     const char *pEnc_String;
15.
16.     connection();
17.     ~connection();
18.     void Check1();
19.     void Check2();
20.     void update();
21.     void establishment();
22.     void execution();
23.
24. };
```

Appendix B-S1: connection.cpp

```
#define __VC
#include "connection.h"
#include "scif.h"
#include "scif_define.h"
#include<string.h>
#include<iostream>
#include <Windows.h> //Sleep(100)


using namespace std;


connection::connection()
{

    DllSetting.SoftwareType = 5;
    DllSetting.TalkInfoNum = 10;
    DllSetting.MemSizeI = I_NUM;
    DllSetting.MemSizeO = O_NUM;
    DllSetting.MemSizeC = C_NUM;
    DllSetting.MemSizeS = S_NUM;
    DllSetting.MemSizeA = A_NUM;
    DllSetting.MemSizeTT = TT_NUM;
    DllSetting.MemSizeCT = CT_NUM;
    DllSetting.MemSizeR = R_NUM;
    DllSetting.MemSizeTS = TS_NUM;
    DllSetting.MemSizeTV = TV_NUM;
    DllSetting.MemSizeCS = CS_NUM;
    DllSetting.MemSizeCV = CV_NUM;
    DllSetting.MemSizeF = F_NUM;

    IP_Address = "192.168.19.200";
    Maker_ID = 23594510;
    pEnc_String = "0B9287F3AE9D949A7751D8C8E51A50BE46FBA406D7E9CE0B";
}

void connection::Check1()
```

```
38. {
39.     //The initialization is the supplier number and the encoded string,
    and used to determine the function is successfully initialized.
40.
41.     int rt = scif_Init(&DllSetting, Maker_ID, pEnc_String);
    //scif_Init(DLL_USE_SETTING *pUseSetting, int MakerID, const char
    *pEncString);
42.     if (rt != 100)
43.     {
44.         printf("Library initialization is failed!");
45.         throw "Unsuccessful connection of in Check1";
46.     }
47. }
48.
49. void connection::Check2()
50. {
51.     //Set the controller IP and determine the connection is established
    successfully
52.     if (scif_LocalConnectIP(0, IP_Address) != 1)
53.     {
54.         printf("Connection Setting is Failed!\n");
55.         throw "Unsuccessful connection of in Check2";
56.     }
57.     else
58.         printf("connected successfully!\n");
59. }
60.
61. void connection::update()
62. {
63.
64.     // Provide alarm, warning, and I/O detect via ReadS(), and define
    ReadR, ReadO, and ReadI must synchronously update the data blocks..
65.
66.     scif_StartCombineSet(0);
67.     //SC_POLLING_CMD:
68.     //This command will be saved in Polling Queue.
69.     //The address set to read will be synchronously updated
70.
```

```
71.     scif_cmd_ReadS(SC_POLLING_CMD, 0, 3000, 4);
72.     scif_cmd_ReadR(SC_POLLING_CMD, 0, 3000, 80);
73.     scif_cmd_ReadR(SC_POLLING_CMD, 0, 6300, 50);
74.     scif_cmd_ReadR(SC_POLLING_CMD, 0, 6000, 80);
75.     scif_cmd_ReadO(SC_POLLING_CMD, 0, 0, 100);
76.     scif_cmd_ReadI(SC_POLLING_CMD, 0, 0, 100);
77.     scif_FinishCombineSet(0);
78. }
79. void connection::establishment()
80. {
81.     //The data are successfully connected after the use of
    scif_GetTalkMsg() is confirmed.
82.     //If it is successfully connected, the connection and the data are
    established.
83.
84.     while (1)
85.     {
86.         if (scif_GetTalkMsg(0, SCIF_CONNECT_STATE) ==
    SC_CONN_STATE_OK)
87.             break;
88.         else
89.             printf("Continue try to be connected... \n");
90.
91.         Sleep(100);
92.     }
93.
94.     printf("data successfully connected!\n");
95. }
96. void connection::execution()
97. {
98.     try { Check1(); }
99.     catch (const char* msg) { cout << msg << endl; }
100.
101.    try { Check2(); }
102.    catch (const char* msg) { cout << msg << endl; }
103.
104.    update();
105.    establishment();
```

```
}

connection::~connection()
{
}
```

Appendix B-H2: MotionPlanning.h

```
#pragma once
class MotionPlanning
{
public:
    // mode 1: find center and give command
    // mode 2: pause the system because robotic arm is running
    // mode 3: Back to the origin
    // mode 4: Back to the origin and stop the software running
    int mode_motion;
    int mode_motion_previous;

    double StartingPoint_World_X;
    double StartingPoint_World_Y;
    double StartingPoint_World_Z;
    double StartingPoint_World_C;

    double destination_World_X;
    double destination_World_Y;

    double detected_World_length;
    double detected_World_width;

    double gripper_height;

    MotionPlanning();
    ~MotionPlanning();

    void Command_from_opencv(double *pointP, double *dp);   // mode 1
    void PauseProgramming(double, double,double,double);   // mode 2
```

```
30.     void BackTo_OriginalPoint(); // mode 3
31.     void CarryToDestination(double,double, double, double);
32.     void Check_OriginalPoint();
33.     double *destination_point;
34.
35. };
```

Appendix B-S2: MotionPlanning.cpp

```
1.  #define __VC
2.  #define _CRT_SECURE_NO_WARNINGS
3.
4.  #include "MotionPlanning.h"
5.  #include<iostream>
6.  #include <scif.h>
7.  #include<Windows.h>
8.  using namespace std;
9.
10. MotionPlanning::MotionPlanning()
11. {
12.
13.
14.
15.     this->gripper_height = 30;
16.
17.     this-> mode_motion=3;
18.     this-> mode_motion_previous = 3;
19.
20. }
21.
22.
23. MotionPlanning::~MotionPlanning()
24. {
25. }
26.
27. void MotionPlanning::Command_from_opencv(double *pointP, double *dp)
    // mode 1
```

```
{

    this->destination_point = dp;
    cout << "Try to send the command line to SCARA ... please wait." << endl;
    scif_cmd_WriteR(0, 8507, 3);
    scif_cmd_WriteR(0, 8503, pointP[1]); // X_color
    scif_cmd_WriteR(0, 8504, pointP[2]); // Y_color
    scif_cmd_WriteR(0, 8505, StartingPoint_World_Z); // Z_up
    scif_cmd_WriteR(0, 17004, 777);
    scif_cmd_WriteC(0, 22, 1);
    cout << "\nWorld Coordinate (x, y) = " << pointP[1] << "\t" << pointP[2] << endl;
    //system("Pause");
    Sleep(2000);

    scif_cmd_WriteR(0, 8505, StartingPoint_World_Z - gripper_height);//z  down
    scif_cmd_WriteR(0, 17004, 777);
    scif_cmd_WriteC(0, 22, 1);
    cout << "\n==========================\n\nGripper off!" << endl;
    Sleep(4000);
    //system("Pause");

    scif_cmd_WriteR(0, 8505, StartingPoint_World_Z );//z up
    scif_cmd_WriteR(0, 17004, 777);
    scif_cmd_WriteC(0, 22, 1);

    this-> mode_motion = 2;
    this-> mode_motion_previous = 1;
    cout << "Command_from_opencv\n" << endl;
    //system("Pause");
    Sleep(3000);
}

void MotionPlanning::PauseProgramming(double ix,double iy,double x,double y)   // mode 2
{
```

```
62.
63.
64.     if (mode_motion_previous == 1)
65.     {
66.         cout << "SCARA is running to grab the coffee cup" << endl;
67.         //cout << "SCARA is moving to the destination" << endl;
68.         scif_cmd_WriteR(0, 8505, StartingPoint_World_Z);//z  down
69.         scif_cmd_WriteR(0, 17004, 777);
70.         scif_cmd_WriteC(0, 22, 1);
71.         CarryToDestination(ix,iy,x,y); // Destination
72.         /*
73.                 scif_cmd_WriteR(0, 8505, StartingPoint_World_Z -
gripper_height);//z up
74.
75.         this-> mode_motion = 3;
76.
77.     }
78.     else if (mode_motion_previous == 3)
79.     {
80.     // Suppose one coffee cup has been lifted
81.     // Please end the programming
82.         cout << "SCARA is moving to the origin" << endl;
83.         this-> mode_motion = 1;
84.     }
85.     this-> mode_motion_previous = 2;
86. }
87. void MotionPlanning::BackTo_OriginalPoint() // mode 3
88. {
89.     // back to origin
90.     cout <<  "Back to origin ... please wait." << endl;
91.     scif_cmd_WriteR(0, 8507, 5);
92.     scif_cmd_WriteR(0, 8503, StartingPoint_World_X); // X_origin
93.     scif_cmd_WriteR(0, 8504, StartingPoint_World_Y); // Y_origin
94.     scif_cmd_WriteR(0, 8505, StartingPoint_World_Z);//z up or down
95.     scif_cmd_WriteR(0, 17004, 777);
96.     scif_cmd_WriteC(0, 22, 1);
97.     cout << "\n\nChange location of the cup and the tape! \n\n" <<
endl;
```

```
98.     Sleep(4000);
99.
100.    this-> mode_motion = 2;
101.    this-> mode_motion_previous = 3;
102. }
103. void MotionPlanning::Check_OriginalPoint()
104. {
105.    scif_cmd_WriteR(0, 8507, 5);
106.    scif_cmd_WriteR(0, 8503, 200); // X_origin
107.    scif_cmd_WriteR(0, 8504, 350); // Y_origin
108.    scif_cmd_WriteR(0, 8505, 220);//z up or down
109.    scif_cmd_WriteR(0, 17004, 777);
110.    scif_cmd_WriteC(0, 22, 1);
111.    cout << "Set origin point" << endl;
112.    //system("Pause");
113.    Sleep(2000);
114. }
115. void MotionPlanning::CarryToDestination(double ix, double iy, double x,
     double y)
116. {
117.    scif_cmd_WriteR(0, 8507, 3);
118.    scif_cmd_WriteR(0, 8503, destination_point[1]);  //x
119.    scif_cmd_WriteR(0, 8504, destination_point[2]);  //y
120.    scif_cmd_WriteR(0, 17004, 777);
121.    scif_cmd_WriteC(0, 22, 1);
122.    std::cout << "Destination here~\n\n" << std::endl;
123.
124.    scif_cmd_WriteR(0, 8505, StartingPoint_World_Z );//z  down
125.    scif_cmd_WriteR(0, 17004, 777);
126.    scif_cmd_WriteC(0, 22, 1);
127.    cout << "\nd1\tPress to lower the gripper\n" << endl;
128.    //system("Pause");
129.    Sleep(2000);
130.
131.    scif_cmd_WriteR(0, 8505, StartingPoint_World_Z -
     gripper_height);//z  down
132.    scif_cmd_WriteR(0, 17004, 777);
133.    scif_cmd_WriteC(0, 22, 1);
```

```
    cout << "\nd2\tPress to increase the altitude of the gripper\n" << endl;
    cout << "\n\n=========================\n\nGripper on!" << endl;
    //system("Pause");
    Sleep(4000);

    scif_cmd_WriteR(0, 8505, StartingPoint_World_Z );//z  down
    scif_cmd_WriteR(0, 17004, 777);
    scif_cmd_WriteC(0, 22, 1);
    cout << "\nd3\tPress to lower the gripper\n" << endl;
    //system("Pause");
    Sleep(2000);
}
```

Appendix B-H3: ReadCoordinate.h

```
#pragma once
class ReadCoordinate
{
public:
    float World_X, World_Y, World_Z, World_C;
    void read_XYZC();
    ReadCoordinate();
    ~ReadCoordinate();
};
```

Appendix B-S3: ReadCoordinate.cpp

```
#include "ReadCoordinate.h"

#define __VC
#include "scif.h"
#include "scif_define.h"
#include<iostream>

ReadCoordinate::ReadCoordinate()
```

```
{
    int Unit_Transform = 100000;
    //float World_X, World_Y, World_Z, World_C;
    World_X = (float)((int)scif_ReadR(6321)) / Unit_Transform;
    World_Y = (float)((int)scif_ReadR(6322)) / Unit_Transform;
    World_Z = (float)((int)scif_ReadR(6323)) / Unit_Transform;
    World_C = (float)((int)scif_ReadR(6326)) / Unit_Transform;
    printf("\nWorldCoordinate\n\t  X = %f  \n\t  Y = %f \n\t  Z = %f \n\t  C = %f \n\n", World_X, World_Y, World_Z, World_C);
}


ReadCoordinate::~ReadCoordinate()
{
}
void ReadCoordinate::read_XYZC()
{
    int Unit_Transform = 100000;
    World_X = (float)((int)scif_ReadR(6321)) / Unit_Transform;
    World_Y = (float)((int)scif_ReadR(6322)) / Unit_Transform;
    World_Z = (float)((int)scif_ReadR(6323)) / Unit_Transform;
    World_C = (float)((int)scif_ReadR(6326)) / Unit_Transform;
}
```

Appendix B-H4: WebCam.h

```
#pragma once
#include <string.h>
#include<opencv2/opencv.hpp>
#include <iostream>

#include "MotionPlanning.h"

using namespace std;

class WebCam
{
```

```
public:

    MotionPlanning mode;
    double StartingPoint_World_X;
    double StartingPoint_World_Y;
    double StartingPoint_World_Z;
    double StartingPoint_World_C;
    double W_x1;  double W_y1;
    double W_x2;  double W_y2;
    double I_x1;  double I_y1;
    double I_x2;  double I_y2;

    double detected_World_length;
    double detected_World_width;

    //Parameter settings
    cv::Mat img;
    cv::Mat img2;
    cv::VideoWriter video;
    cv::VideoCapture cap;
    int frame_width, frame_height;
    double Point_original_opencv[4];  //[x, y, radius]
    double Point_processed_opencv[4]; //[x, y, radius]
    double Point_processed_destination_opencv[4]; //[x, y, radius]

    double min_radius_cup_detected;
    double min_radius_destination_detected;

    int camera_number = 1;

    string filename;
    int fps = 10;// frames per second

    int Gaussian_sigmaX = 2; //Gaussian kernel standard deviation in X direction.
    int Gaussian_sigmaY = 2; //Gaussian kernel standard deviation in Y direction.
    int HoughCircles_parameter1 = 200;
```

```
    int HoughCircles_parameter2 = 100;
    int dp = 1; // Inverse ratio of the accumulator resolution to the image resolution.For example, if dp = 1, the accumulator has the same resolution as the input image.
    int HoughCircles_min_radius = 0;
    int HoughCircles_max_radius = 0;

    // mode 1: find center and give command
    // mode 2: pause the system because robotic arm is running
    // mode 3: Back to the origin

    int mode_motion;

    WebCam(double, double, double, double);
    ~WebCam();

    void CheckVideo_isOpened();
    void execution();
    void GetFrameResolution();
    void WhileLoop();
    int BreakWhile();
    void Release_video();
};
```

Appendix B-S4: WebCam.cpp

```
#include "WebCam.h"
#include "MotionPlanning.h"

#include <iostream>
#include <string.h>
#include<opencv2/opencv.hpp>
using namespace std;

#define offset_x 5
#define offset_y 15
#define offset_x_1 10
```

```
12. #define offset_y_1 10
13. WebCam::WebCam(double x, double y, double z, double c)
14. {
15.     //Parameter settings
16.     this->StartingPoint_World_X = x;
17.     this->StartingPoint_World_Y = y;
18.     this->StartingPoint_World_Z = z;
19.     this->StartingPoint_World_C = c;
20.
21.     MotionPlanning mode;
22.     this-> mode.StartingPoint_World_X = x;
23.     this-> mode.StartingPoint_World_Y = y;
24.     this-> mode.StartingPoint_World_Z = z;
25.     this-> mode.StartingPoint_World_C = c;
26.     double deviation_SCARA_x = -0;
27.     double deviation_SCARA_y = 150;
28.     this->detected_World_length = 150;  mode.detected_World_length =
    this->detected_World_length;
29.     this->detected_World_width = 100;    mode.detected_World_width =
    this->detected_World_width;
30.
31.     this->  W_x1 = x- deviation_SCARA_x; this-> W_y1 = y+
    deviation_SCARA_y;
32.     this->  W_x2 = x- deviation_SCARA_x - detected_World_length;
        this->W_y2 = y+ deviation_SCARA_y + detected_World_width;
33.
34.     this-> I_x1 = 498; this->I_y1 = 214;  // (x1, y1)  //   center =
    [498, 214] ; radius = 63
35.     this->I_x2 = 284;  this->I_y2 = 84; // (x2, y2)  //center = [284,
    84] ; radius = 59
36.     this-> min_radius_cup_detected = 60;
37.     this->min_radius_destination_detected = 58;
38.
39.     this->camera_number = 1;
40.     cv::VideoCapture cap1(camera_number + cv::CAP_DSHOW);
41.     cap = cap1;
42.
43.     this-> filename = "OpenCV_processed.avi";
```

```
44.     this->fps = 10; // frames per second
45.     this->Gaussian_sigmaX = 2; //Gaussian kernel standard deviation in
X direction.
46.     this->Gaussian_sigmaY = 2; //Gaussian kernel standard deviation in
Y direction.
47.     this->HoughCircles_parameter1 = 200;
48.     this->HoughCircles_parameter2 = 100;
49.     this->dp = 1; // Inverse ratio of the accumulator resolution to
the image resolution.For example, if dp = 1, the accumulator has the
same resolution as the input image.
50.     this->HoughCircles_min_radius = 0;
51.     this->HoughCircles_max_radius = 0;
52. }
53.
54. WebCam::~WebCam()
55. {
56. }
57.
58. void WebCam::execution()
59. {
60.
61.     try { CheckVideo_isOpened(); }
62.     catch (const char* msg) { cout<< msg << endl; }
63.     cv::namedWindow("main", CV_WINDOW_AUTOSIZE); //Creates a window.
64.     GetFrameResolution();
65.
66.     cv::VideoWriter video1(filename, CV_FOURCC('M', 'J', 'P', 'G'),
fps, cv::Size(frame_width, frame_height));
67.     video = video1;
68.     WhileLoop();
69.     Release_video();
70.
71. }
72.
73. void WebCam::CheckVideo_isOpened()
74. {
75.     if (!cap.isOpened())
76.     {
```

```
77.         cout << "Change the camera port number!\n";
78.         throw "Unsuccessful video connection";
79.     }
80. }
81.
82. void WebCam::GetFrameResolution()
83. {
84.     // Default resolution of the frame is obtained.The default
    resolution is system dependent.
85.     this->frame_width = cap.get(CV_CAP_PROP_FRAME_WIDTH); // Width of
    the frames in the video stream.
86.     this->frame_height = cap.get(CV_CAP_PROP_FRAME_HEIGHT); // Height
    of the frames in the video stream.
87. }
88.
89. void WebCam::WhileLoop()
90. {
91.     mode.mode_motion = 1;
92.
93.     while (mode.mode_motion != 4)
94.     {
95.         cap >> img;
96.
97.         cvtColor(img, img2, CV_BGR2GRAY); //The conversion from a RGB
    image to gray
98.
99.
100.        // GaussianBlur
101.        //Blurs an image using a Gaussian filter.
102.    //https://docs.opencv.org/2.4/modules/imgproc/doc/filtering.html?h
    ighlight=gaussianblur#gaussianblur
103.        GaussianBlur(img2, img2, cv::Size(9, 9), this-
    >Gaussian_sigmaX, this->Gaussian_sigmaY); //(input_Image, output_Image,
    Gaussian kernel size, sigmaX, sigmaY, borderType )
104.        //GaussianBlur(InputArray src, OutputArray dst, Size ksize,
    double sigmaX, double sigmaY=0, int borderType=BORDER_DEFAULT )
105.
106.
```

```
107.         double min_distance = img2.rows / 8;  // Minimum distance
between the centers of the detected circles.
108.         vector <cv::Vec3f> circles;
109.         int GetCircle = 0; // There is no circle in the beginning.
110.
111.         // HoughCircles
112.         //
113.         // Finds circles in a grayscale image using the Hough
transform.
114.         //void HoughCircles(InputArray image, OutputArray circles,
int method, double dp, double minDist, double param1 = 100, double
param2 = 100, int minRadius = 0, int maxRadius = 0)
115.     //https://docs.opencv.org/2.4/modules/imgproc/doc/feature_detectio
n.html?highlight=houghcircles
116.         HoughCircles(img2, circles, CV_HOUGH_GRADIENT, this->dp,
min_distance, this->HoughCircles_parameter1, this-
>HoughCircles_parameter2, this->HoughCircles_min_radius, this-
>HoughCircles_min_radius);
117.
118.
119.         Point_original_opencv[1] = 0;
120.         for (size_t i = 0; i < circles.size(); i++)
121.         {
122.             cv::Point center(cvRound(circles[i][0]),
cvRound(circles[i][1]));
123.             float radius = cvRound(circles[i][2]);
124.
125.             if (radius > min_radius_cup_detected) //cup
126.             {
127.                 Point_original_opencv[1] = center.x;
128.                 Point_original_opencv[2] = center.y;
129.                 Point_original_opencv[3] = radius;
130.
131.                 Point_processed_opencv[1] = W_x1 + (W_x2 -
W_x1)*(Point_original_opencv[1] - I_x1) / (I_x2 - I_x1);
132.                 Point_processed_opencv[2] = W_y1 + (W_y2 -
W_y1)*(Point_original_opencv[2] - I_y1) / (I_y2 - I_y1);
133.                 Point_processed_opencv[3] = Point_original_opencv[3]
```

```
* (Point_original_opencv[1] - I_x1) / (I_x2 - I_x1);

                    cout << "cup\tOriginal: center.x = " << Point_original_opencv[1] << " ; center.y = " << Point_original_opencv[2] << " ; radius = " << Point_original_opencv[3] << endl;
                    GetCircle = 1; // We get at least one circle!

                }
                else if (radius < min_radius_cup_detected) //destination
                {
                    Point_original_opencv[1] = center.x;
                    Point_original_opencv[2] = center.y;
                    Point_original_opencv[3] = radius;

                    Point_processed_destination_opencv[1] = W_x1 + (W_x2 - W_x1)*(Point_original_opencv[1] - I_x1) / (I_x2 - I_x1);
                    Point_processed_destination_opencv[2] = W_y1 + (W_y2 - W_y1)*(Point_original_opencv[2] - I_y1) / (I_y2 - I_y1);
                    Point_processed_destination_opencv[3] = Point_original_opencv[3] * (Point_original_opencv[1] - I_x1) / (I_x2 - I_x1);

                    cout << "\t tape\tOriginal: center.x = " << Point_original_opencv[1] << " ; center.y = " << Point_original_opencv[2] << " ; radius = " << Point_original_opencv[3] << endl << endl;
                }
                circle(img, center, 3, cv::Scalar(0, 255, 0), -1, 8, 0); // draw the circle center
                circle(img, center, radius, cv::Scalar(0, 0, 255), 3, 8, 0); // draw the circle outline
            }

            //Point_processed_destination_opencv[1] -= offset_x;
            Point_processed_opencv[1] += offset_x;
            //Point_processed_destination_opencv[2] -= offset_y;
            Point_processed_opencv[2] += offset_y;
```

```
159.
160.         //video.write(img);  // Write the frame into the file
'outcpp.avi'
161.         imshow("main", img);
162.         if (BreakWhile() < 0) break;
163.
164.         //=====================================================
165.         // Motion Planning
166.
167.         cout << "mode.mode_motion = " << mode.mode_motion << endl;
168.
169.         if (mode.mode_motion == 1 && GetCircle == 1)
170.         {
171.             if (Point_original_opencv[1] <= 0)
172.                 continue;
173.
174.             mode.Command_from_opencv(Point_processed_opencv,
Point_processed_destination_opencv);
175.             GetCircle = 0;
176.         }
177.         else if (mode.mode_motion == 1 && GetCircle == 0)
178.             continue;
179.         else if (mode.mode_motion == 2)
180.             mode.PauseProgramming(this->StartingPoint_World_X, this-
>StartingPoint_World_Y, this->detected_World_length, this-
>detected_World_width);
181.         else if (mode.mode_motion == 3)
182.             mode.BackTo_OriginalPoint();
183.         else if (mode.mode_motion == 4)
184.             break;
185.         //
186.         //=====================================================
187.
188.     } //end while
189.
190. }
191. void WebCam::Release_video()
192. {
```

```
193.
194.     // When everything done, release the video capture and write
object
195.     cap.release();
196.     video.release();
197.
198.     // Closes all the windows
199.     cv::destroyAllWindows();
200.     cout << "\nThe other windows are all destroyed" << endl;
201. }
202. int WebCam::BreakWhile()
203. {
204.     char c = (char)cv::waitKey(1);
205.     if (c == 27)
206.         return -1;
207.     else if (c == 112)
208.     {
209.         system("Pause");
210.         cout << "\nVideo continued" << endl;
211.         return 1;
212.     }
213.     else return 1;
214. }
```